

# IFT436 - Devoir #1

À remettre avant le **vendredi 25 septembre avant 23h59**.  
Tout retard entraînera jusqu'à 33% de pénalité par jour.

**Pondération.** Ce travail compte pour 6% des points de la session.

**Modalités.** Considérez les points suivants.

- Le travail peut être fait seul ou en équipe de deux.
- Le devoir doit être remis en format pdf via turnin. La provenance de votre pdf n'a pas d'importance — ce pdf peut provenir d'un export d'un fichier Word ou d'un fichier texte, d'un scan de vos écrits/dessins, etc.  
Il n'y a pas de norme de présentation particulière. Écrivez lisiblement, et assurez-vous seulement que **vos noms et CIP apparaissent clairement sur le devoir remis**.
- Toute argumentation qui démontre l'énoncé correctement sera acceptée, que ce soit sous la forme de série formules mathématiques, ou bien d'explications textuelles (ou un mélange des deux).
- Vous pouvez utiliser les techniques de preuves que vous désirez, vous n'êtes pas dans l'obligation de suivre les suggestions (mais je le recommande fortement).
- Vous pouvez utiliser les algorithmes et résultats démontrés en classe et dans les solutionnaires d'exercices sans avoir à les démontrer. Par exemple, si vous voulez utiliser le tri par insertion, vous pouvez l'invoquer sans écrire le pseudo-code de l'algorithme. Par contre, si vous utilisez de nouveaux résultats, vous devez les démontrer.

### Question 1 : une question d'échauffement (15 points)

Dans le problème de la *sélection*, on veut trouver la  $k$ -ième plus petite valeur dans un tableau donné contenant des valeurs distinctes (le tableau n'est pas nécessairement trié).

#### Problème de la sélection

**Entrée:** un tableau  $T$  d'entiers contenant  $n$  éléments distincts, et un entier  $k \in \{1, 2, \dots, n\}$ .

**Sortie:** un indice  $p \in \{1, 2, \dots, n\}$  tel que

$$|\{q : 1 \leq q \leq n \wedge T[q] < T[p]\}| = k - 1.$$

Par exemple, si  $T = [7, 9, 0, 5, 10, 2, 11]$  et  $k = 3$ , il faut retourner 4 parce que  $T[4] = 5$  est la troisième plus petite valeur (il y a 0 et 2 qui sont plus petits que 5). Autre exemple, si  $k = 5$  alors il faut retourner 2 car  $T[2] = 9$ .

Donnez le pseudo-code d'un algorithme qui résout le problème de la sélection. Donnez ensuite la complexité en temps de votre algorithme avec la notation  $O$ , sans justification.

Vous aurez un maximum de points pour un algorithme en temps  $O(n \log n)$ . Vous n'avez pas à justifier que votre algorithme est correct.

## Question 2 : le problème de la sous-somme (50 points)

Dans le problème de la sous-somme, on a un ensemble d'entiers et on veut savoir si on peut trouver un sous-ensemble d'éléments qui somment à  $t$ .

### Problème de la sous-somme

**Entrée:** un ensemble  $S$  d'entiers positifs et un entier  $t > 0$ .

**Sortie:** un sous-ensemble  $S' \subseteq S$  tel que  $\sum_{s \in S'} s = t$ , ou bien *null* s'il n'existe pas de tel  $S'$ .

a. Considérez l'algorithme suivant.

```
fonction algo1( $S, t$ )
    si  $\sum_{s \in S} s < t$  alors return null;
     $q = 0$ ;
     $S' = \emptyset$ ;
    tant que  $q < t$  faire
        Soit  $m$  l'élément maximum de  $S$ ;
         $q = q + m$ ;
        Retirer  $m$  de  $S$ ;
        Ajouter  $m$  à  $S'$ ;
    fin
    si  $q == t$  alors
        return  $S'$ ;
    sinon
        return null;
    fin
```

Est-ce que cet algorithme résout correctement le problème de la sous-somme? Démontrez votre réponse.

b. Considérez l'algorithme suivant.

```
fonction algo2( $S, t$ )
    pour chaque sous-ensemble  $S'$  de  $S$  faire
        si les éléments de  $S'$  somment à  $t$  alors
            return  $S'$ ;
        fin
    fin
    return null;
```

Cette question comporte deux sous-questions:

(1) Est-ce que cet algorithme résout correctement le problème de la sous-somme? Démontrez votre réponse.

(2) Combien de fois est-ce que l'algorithme entrera dans la boucle **pour chaque**, dans le pire cas en fonction de  $n = |S|$ ? Justifiez votre réponse.

- c. Considérez maintenant la variante suivante du problème de la sous-somme, dans laquelle on nous spécifie la taille du sous-ensemble désirée.

**Problème de la sous-somme de taille désirée**

**Entrée:** un ensemble  $S$  d'entiers positifs, un entier  $k \in \{0, 1, \dots, |S|\}$  et un entier  $t > 0$ .

**Sortie:** un sous-ensemble  $S' \subseteq S$  tel que  $|S'| = k$  et tel que  $\sum_{s \in S'} s = t$ , ou bien *null* s'il n'existe pas de tel  $S'$ .

L'algorithme ci-haut peut être modifié pour énumérer seulement les sous-ensembles de la taille voulue.

```
fonction algo3( $S, t, k$ )  
    pour chaque sous-ensemble  $S'$  de  $S$  tel que  $|S'| = k$  faire  
        si les éléments de  $S'$  somment à  $t$  alors  
            return  $S'$ ;  
        fin  
    fin  
    return null;
```

Cette question comporte deux sous-questions:

- (1) Combien de fois est-ce que l'algorithme entrera dans la boucle **pour chaque**, dans le pire cas en fonction de  $k$  et  $n = |S|$ ? Justifiez votre réponse.
- (2) Est-ce que la valeur de votre réponse en c.(1) est plus petite que la valeur de votre réponse en b.(2), pour  $n > 0$ ? Justifiez votre réponse.

### Question 3: puissances en temps logarithmique (35 points)

Soient  $x$  et  $k$  deux entiers tels que  $x \geq 1$  et  $k \geq 0$ . La fonction  $\text{pow}(x, k)$  retourne la valeur de  $x^k$ . Elle est implémentée par l'algorithme suivant:

```
fonction  $\text{pow}(x, k)$ 
|   si  $k == 0$  alors
|   |   Retourner 1;
|   sinon si  $k == 1$  alors
|   |   Retourner  $x$ ;
|   sinon
|   |    $\text{tmp} = \text{pow}(x, \lfloor k/2 \rfloor)$  ;           //  $\lfloor k/2 \rfloor$  = valeur plancher de  $k/2$ 
|   |   si  $k \% 2 == 0$  alors
|   |   |   Retourner  $\text{tmp} * \text{tmp}$ ;
|   |   sinon
|   |   |   Retourner  $\text{tmp} * \text{tmp} * x$ ;
|   fin
fin
```

Rappelons que  $\%$  est le *modulo*, c'est-à-dire le reste de la division entière. On a  $k \% 2 == 0$  si et seulement si  $k$  est un nombre pair.

Cet algorithme est intéressant, car le nombre d'appels total est logarithmique en  $k$ .

- a. Démontrez que cet algorithme calcule correctement  $x^k$ .

*Suggestion:* par induction sur  $k$ , commencez avec  $k = 0$  et  $k = 1$  comme cas de base.

- b. Soit  $r(k)$  le nombre d'appels à la fonction effectués lors de l'exécution, en comptant l'appel initial. Par exemple,  $r(1) = 1$  et  $r(4) = 3$ . Nous allons supposer que  $k$  est une puissance de 2.

Montrez de façon rigoureuse que pour tout  $k \in \mathbb{N}$  tel que  $k$  est une puissance de 2, on a  $r(k) = \log(k) + 1$ .

*Suggestion:* Représentez  $r(k)$  par une récurrence et résolvez cette dernière par induction sur les exposants des puissances de 2, avec  $k = 2^0 = 1$  comme cas de base.